

■ Lab 11 — Déployer Nginx avec user_data

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Déployé un serveur **Nginx** sur une instance EC2 avec **user_data** / cloud-init
- Compris pourquoi user_data est préféré aux provisioners
- Utilisé `terraform apply -replace` pour forcer la recréation d'une ressource
- Vérifié le déploiement avec `curl`

■ **Pourquoi user_data plutôt qu'un provisioner ?** Le provisioner `remote-exec` requiert une connexion SSH sortante depuis Terraform — fragile en CI/CD. `user_data` s'exécute côté AWS au démarrage de l'instance, sans aucune dépendance réseau côté Terraform. C'est l'approche recommandée par HashiCorp.

■ Étape 1 — Créer les fichiers

```
cd labs/lab11-provisioners
```

``backend.tf``

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region     = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ Remplacez `<votre-prenom>` par votre prénom. Ex : `tf-training-alice-982908300187`

``versions.tf``

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

`variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}
```

`locals.tf`

```
locals {  
  prefix = "lab11-${var.username}"  
  
  # Script user_data : s'exécute au démarrage de l'instance, côté AWS  
  # Aucune connexion SSH requise depuis Terraform  
  user_data = <<-EOF  
    #!/bin/bash  
    dnf update -y  
    dnf install -y nginx  
    systemctl start nginx  
    systemctl enable nginx  
    echo "<h1>Lab 11 - ${var.username}</h1>" > /usr/share/nginx/html/index.html  
  EOF  
  
  common_tags = {  
    Lab          = "lab11"  
    Username     = var.username  
    ManagedBy    = "Terraform"  
  }  
}
```

`main.tf`

```

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "lab11_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 11 - ${var.username}"

  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
    description  = "HTTP"
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab11_instance" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab11_sg.id]
  associate_public_ip_address = true

  # user_data s'exécute une seule fois au démarrage de l'instance
  # Terraform n'a pas besoin de connexion SSH ou réseau vers l'instance
  user_data = local.user_data

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-nginx"
  })
}

```

outputs.tf`

```
output "instance_public_ip" {
  description = "IP publique de l'instance Nginx"
  value       = aws_instance.lab11_instance.public_ip
}

output "nginx_url" {
  description = "URL pour accéder à Nginx"
  value       = "http://${aws_instance.lab11_instance.public_ip}"
}
```

■ Étape 2 — Déployer

```
terraform init
terraform apply -var="username=<votre-prenom>"
```

■ Le déploiement prend 1-2 minutes. Attendre ensuite **30 à 60 secondes** le temps que cloud-init termine l'installation de Nginx.

■ Étape 3 — Vérifier Nginx

```
NGINX_IP=$(terraform output -raw instance_public_ip)
curl http://$NGINX_IP
```

Résultat attendu :

```
<h1>Lab 11 - <votre-prenom></h1>
```

■ Étape 4 — Forcer la recreation avec -replace

Modifiez le message dans `locals.tf` :

```
echo "<h1>Lab 11 - ${var.username} - v2</h1>" > /usr/share/nginx/html/index.html
```

■ Modifier `user_data` seul ne recrée pas l'instance. Il faut utiliser `-replace` :

```
terraform apply -replace="aws_instance.lab11_instance" -var="username=<votre-prenom>"
```

Vérifiez après redémarrage (~60 secondes) :

```
NGINX_IP=$(terraform output -raw instance_public_ip)
curl http://$NGINX_IP
```

Résultat attendu :

```
<h1>Lab 11 - <votre-prenom> - v2</h1>
```

■ `-replace` est la commande moderne qui remplace `terraform taint` (dépréciée depuis v0.15.2).

■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

■ Validation du Lab

#	Critère	Validé
1	terraform apply réussit sans clé SSH ni connexion distante	■
2	curl http://<IP> retourne la page HTML Lab 11	■
3	-replace force la recréation de l'instance	■
4	curl retourne la page mise à jour (v2) après recréation	■
5	terraform destroy supprime toutes les ressources	■

■ Résolution des problèmes courants

Problème	Cause	Solution
curl renvoie une erreur de connexion	cloud-init pas encore terminé	Attendre 60s et relancer curl
Page HTML non mise à jour	user_data ne se ré-exécute pas automatiquement	Utiliser -replace pour recréer l'instance
Connection refused port 80	Security Group mal configuré	Vérifier le port 80 dans le SG

■ Fin du Lab 11 — Nginx est déployé sans provisioner, via user_data !

> Le Lab 12 introduit les **modules** pour organiser et réutiliser votre code Terraform.